Bölüm 12
İstisna Yönetimi ve Metin I/O
(Exception Handling and Text IO)



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Motivasyon

Bir program, çalışma zamanı hatasıyla (runtime error) karşılaştığında, program anormal olarak sona erer. Programın düzgün bir şekilde çalışmaya veya sonlandırmaya devam edebilmesi için çalışma zamanı hatasını nasıl ele alabiliriz? Bu bölümde tanıtacağımız konu budur.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Amaçlar

- İstisnalar ve istisna işleme hakkında genel bilgi edinmek (12.2).
- İstisna yönetimi kullanımının avantajlarını keşfetme (12.2).
- İstisna tiplerini ayırdetme: **Error** (fatal) - **Exception** (nonfatal) ve kontrollü-kontrolsüz (12.3).
- Metod başlığında istisna bildirimi (12.4.1).
- İstisnaları atmak (12.4.2).
- İstisna yönetimi için **try-catch** bloğu yazma (12.4.3).
- İstisnanın nasıl yayıldığını açıklama (12.4.3).
- İstisna objesinden bilgi alma (12.4.4).
- İstisna yönetimi ile uygulama geliştirmme (12.4.5).
- **try-catch** bloğu içinde **finally** kullanma (12.5).
- Sadece beklenmedik hatalar için istisna kullanımı (12.6).
- **catch** bloğu içinden istisnayı yeniden atma (12.7).
- Zincirleme istisna (12.8).
- Özel istisna sınıfları tanımlamak (12.9).
- Dosya / dizin özelliklerini keşfetmek, dosyaları / dizinleri silmek ve yeniden adlandırmak ve File sınıfını kullanarak dizinler oluşturmak (12.10).
- **PrintWriter** sınıfı kullanarak bir dosyaya veri yazma (12.11.1).
- Kaynakların otomatik olarak kapatılmasını sağlamak için kaynaklarla deneme özelliğini kullanmak (12.11.2).
- **Scanner** sınıfını kullanarak bir dosyadan veri okuma (12.11.3).
- **Scanner** kullanılarak nasıl data okunduğunu anlama (12.11.4).
- Bir dosyanın içindeki bir metni değiştirme (12.11.5).
- Webden veri okuma (12.12).
- Web crawler geliştirme (12.13).



İstisna-Yönetimine genel bakış

runtime hatası göster

Quotient Run

if statement kullanarak düzeltme

QuotientWithIf Run

Metod ile

QuotientWithMethod Run



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

İstisna Avantajları

QuotientWithException Run

Örnek istisna yönetiminin avantajını göstermektedir. Bir yöntemi, arayanın için bir istisna sağlar Bu özellik olmadan, bir metod istisna yönetebilmeli veya programı sonlandırabilmelidir.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

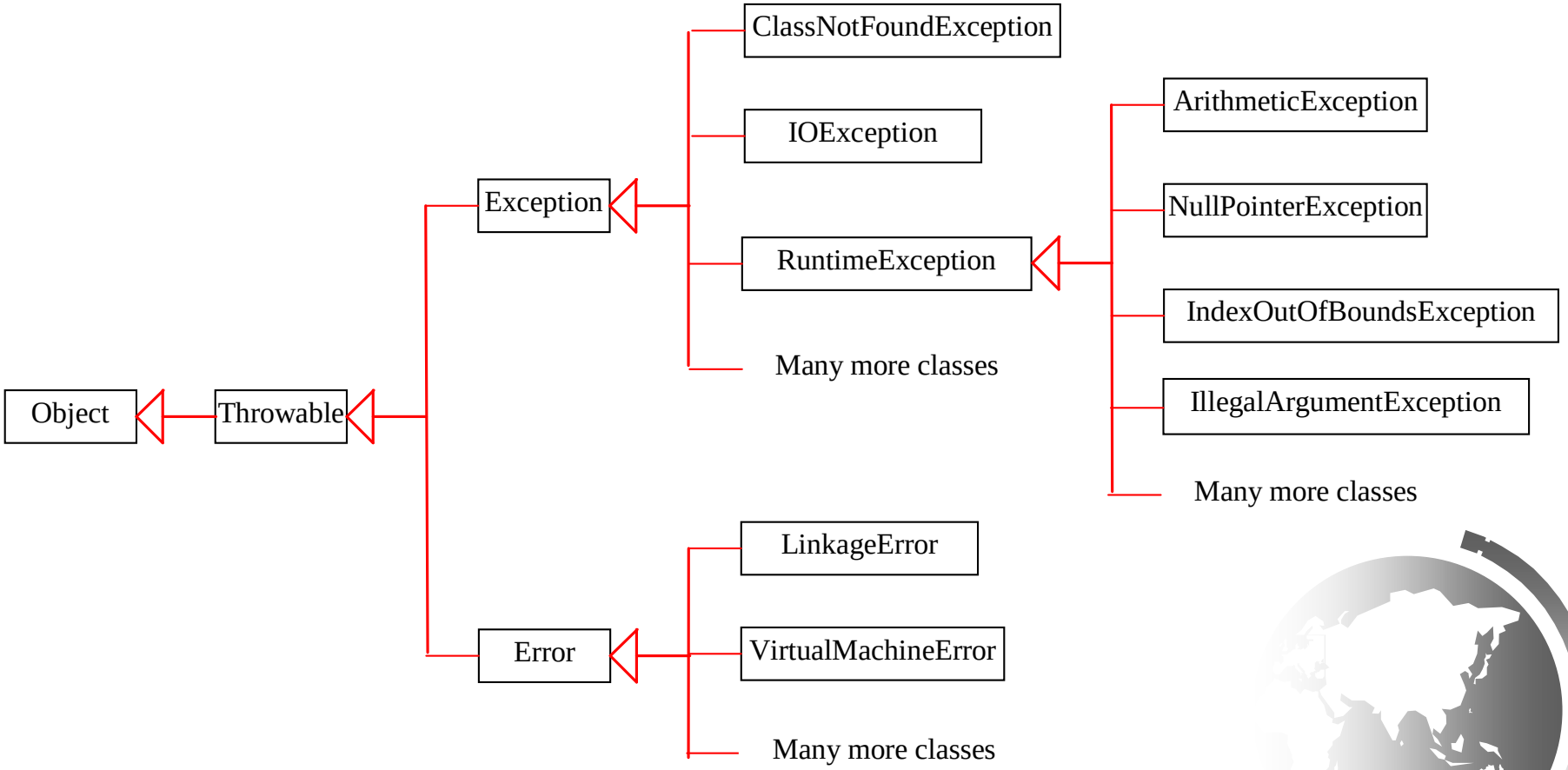
InputMismatchException

InputMismatchExceptionDemo Run

InputMismatchException ile, programınız sürekli doğru olana kadar bir giriş okuyacaktır.

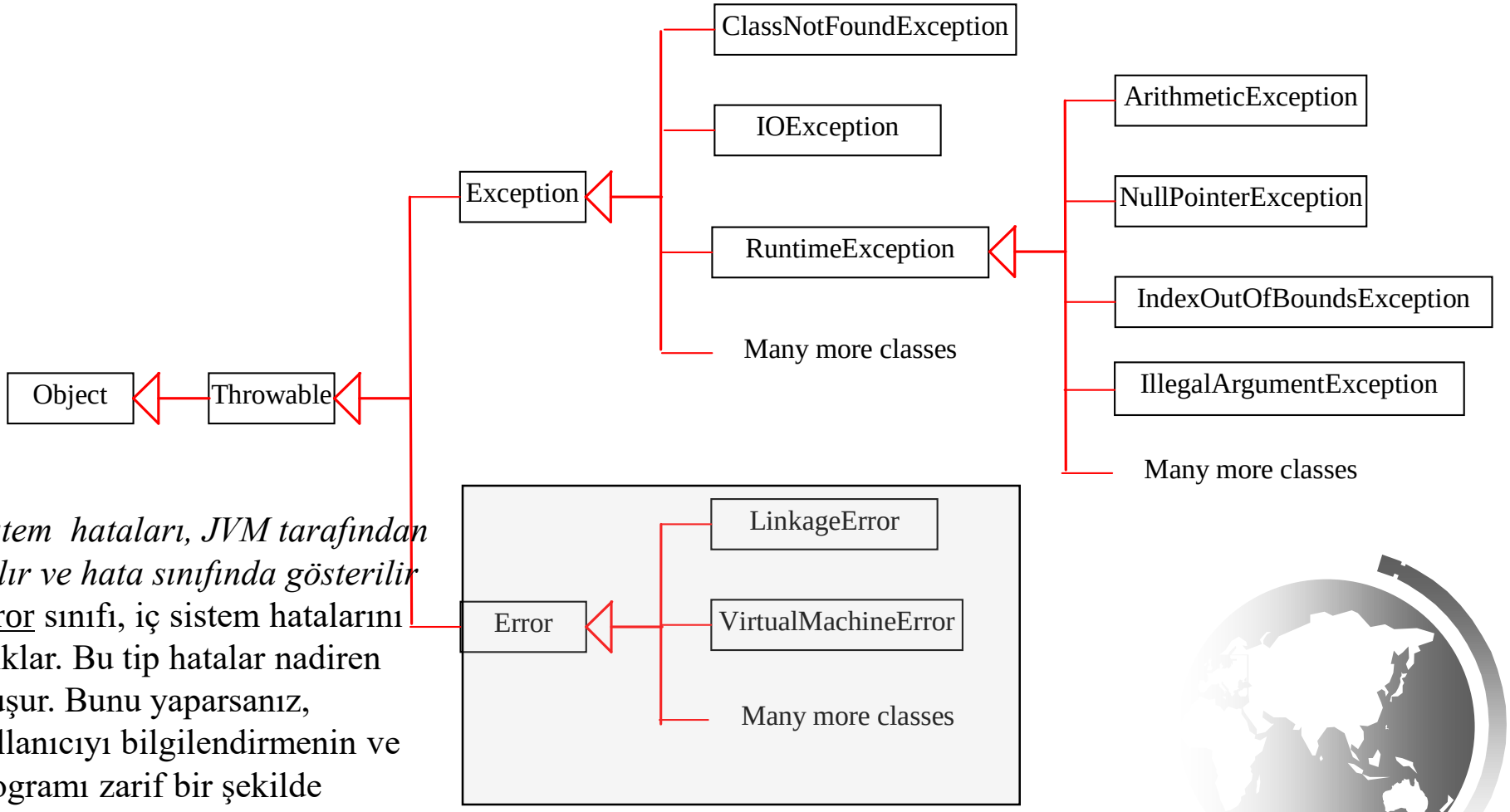


Exception Türleri



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Sistem Hataları



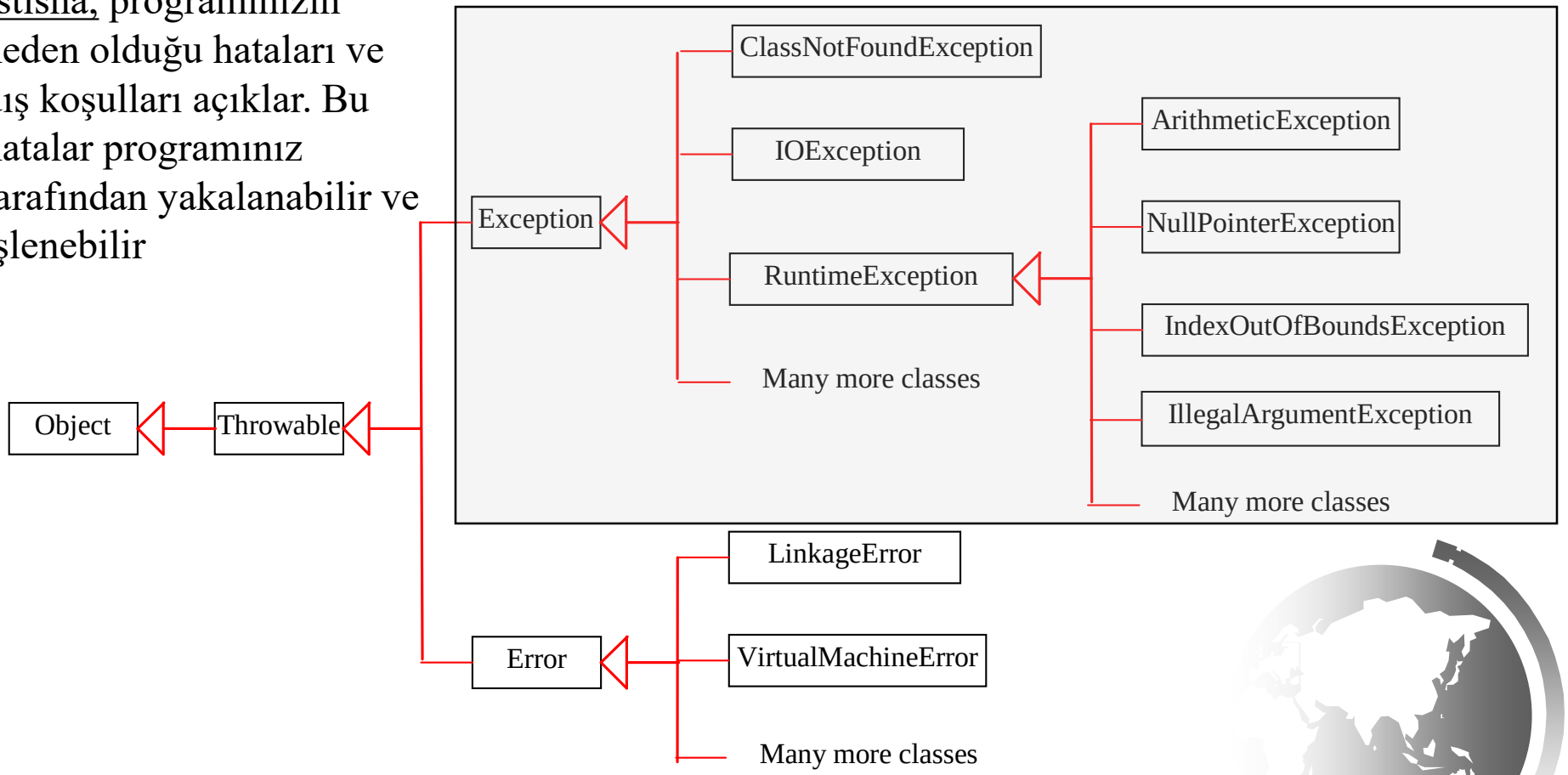
Sistem hataları, JVM tarafından atılır ve hata sınıfında gösterilir

Error sınıfı, iç sistem hatalarını açıklar. Bu tip hatalar nadiren oluşur. Bunu yaparsanız, kullanıcıyı bilgilendirmenin ve programı zarif bir şekilde sonlandırmanın ötesinde yapabileceğiniz çok az şey vardır

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

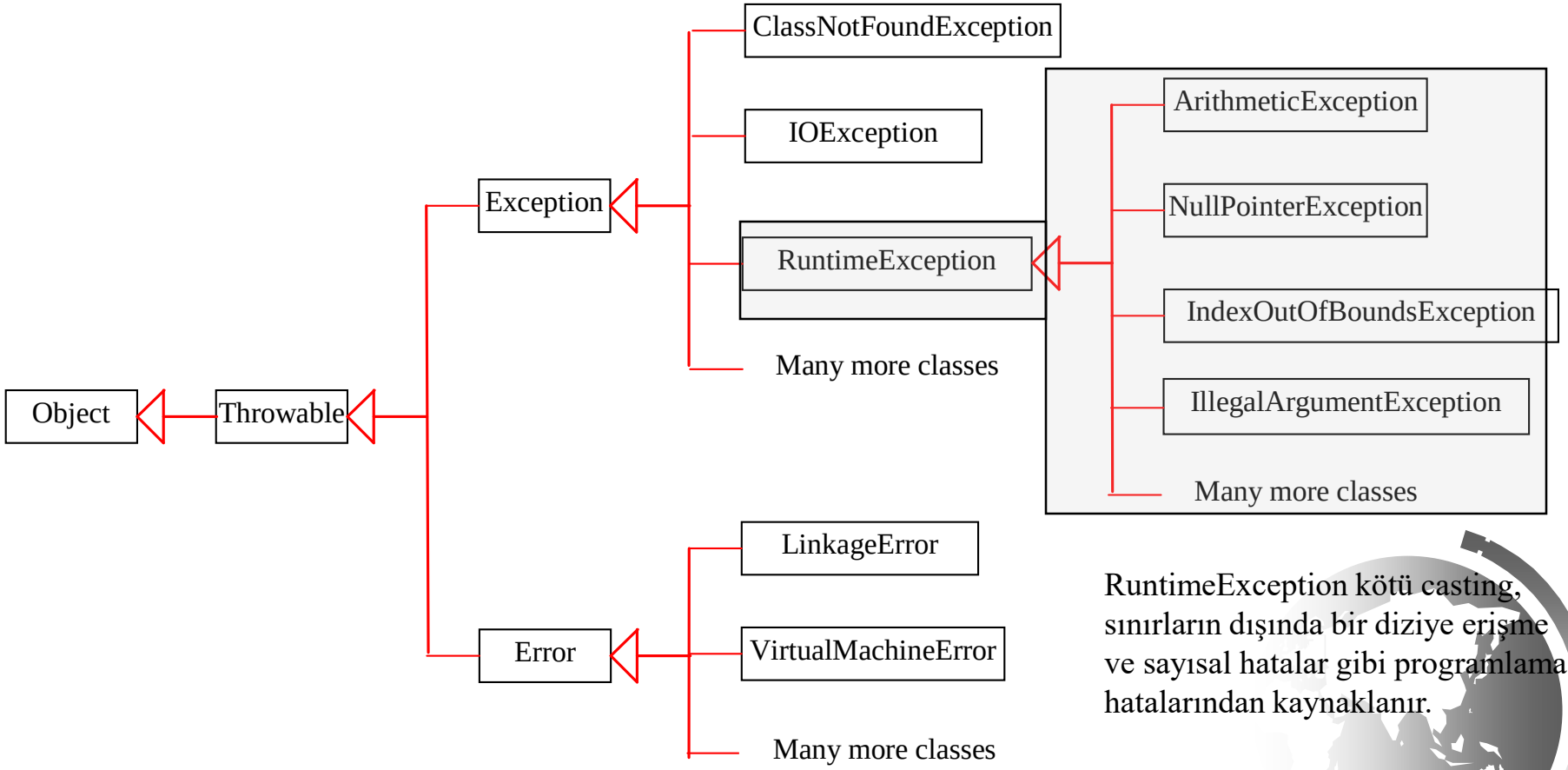
İstisnalar

İstisna, programınızın neden olduğu hataları ve dış koşulları açıklar. Bu hatalar programınız tarafından yakalanabilir ve işlenebilir



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Runtime Exceptions



RuntimeException kötü casting, sınırların dışında bir diziye erişme ve sayısal hatalar gibi programlama hatalarından kaynaklanır.

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Kontrollü ve Kontrolsüz İstisnalar

RuntimeException, Error ve alt sınıfları kontrolsüz istisnalar olarak bilinir. Diğer tüm istisnalar kontrollü istisnalar olarak bilinir, yani derleyici programcıyı istisnaları kontrol etmeye ve ele almaya zorlar.



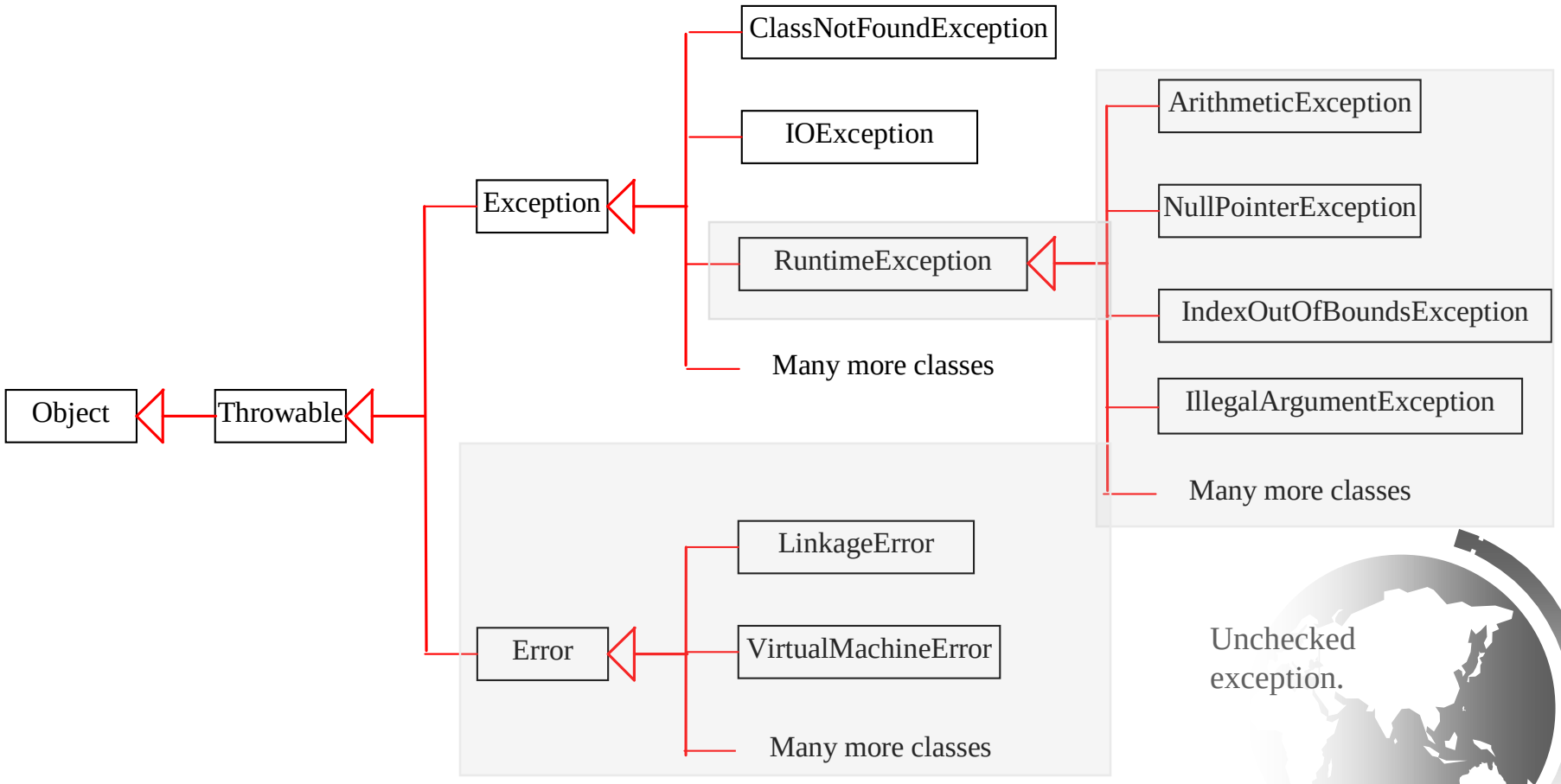
Kontrolsüz İstisnalar

Çoğu durumda, kontrolsüz istisnalar, kurtarılamayan programlama mantığı hatalarını yansıtır. Örneğin, bir nesneye bir nesne atanmadan önce bir referans değişkeni üzerinden erişirseniz, NullPointerException oluşturulur; Dizinin sınırları dışındaki bir dizideki öğeye erişirseniz, IndexOutOfBoundsException oluşturulur. Bunlar program içerisinde düzeltilmesi gereken mantık(logic) hatalarıdır. Programın herhangi bir yerinde kontrolsüz istisnalar oluşabilir. Try-catch bloklarının hantal kullanımından kaçınmak için Java, kontrolsüz istisnaları yakalamak için kod yazmanıza izin vermez.



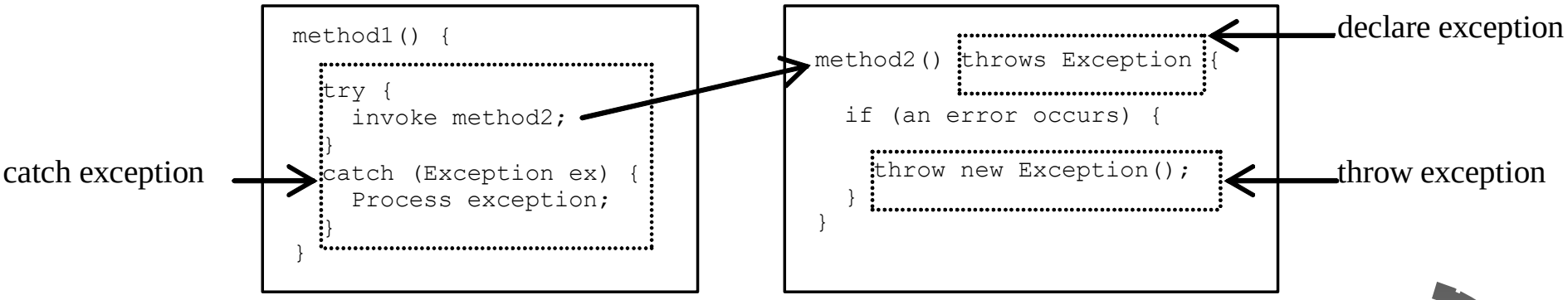
Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Kontrolsüz İstisnalar



Unchecked exception.

İstisna bildirimi(declaring), atma (throwing) ve yakalama (catching)



İstisna bildirimi

Her yöntem, atabileceği kontrollü istisna türlerini belirtmelidir. Buna *istisna bildirme* denir.

```
public void myMethod()  throws IOException

public void myMethod()
   throws IOException, OtherException
```



İstisna Atma

Program bir hata algıladığında, program uygun bir istisna türünün bir örneğini oluşturabilir ve atabilir. Bu *istisna atma* olarak bilinir. Örneğin,

```
throw new TheException();
```

```
TheException ex = new TheException();
throw ex;
```



Örnek: İstisna atma

```
   /** Set a new radius */
public void setRadius(double newRadius)
    throws IllegalArgumentException {
  if (newRadius >= 0)
    radius =  newRadius;
  else
    throw new IllegalArgumentException(
      "Radius cannot be negative");
}
```



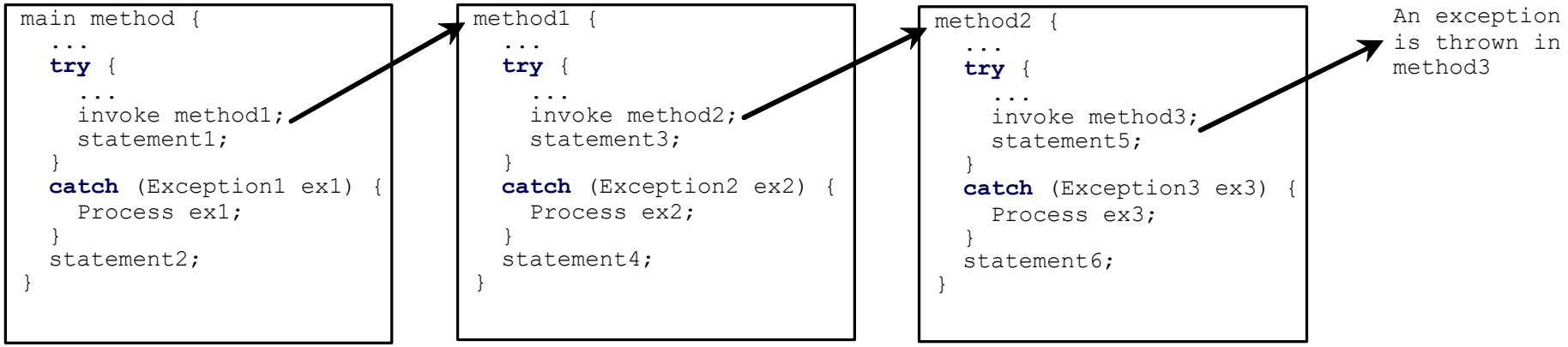
İstisna Yakalama

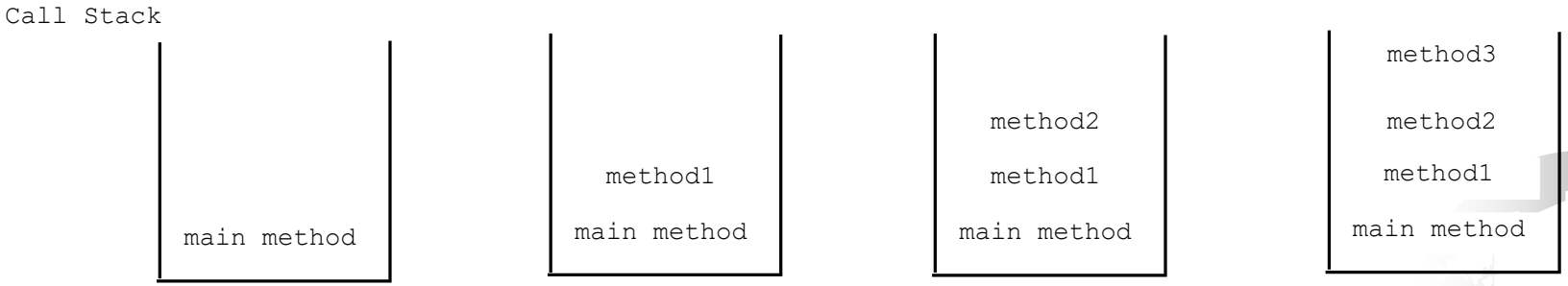
```
try {
  statements;  // İstisna atabilecek ifadeler
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

İstisna Yakalama





Kontrollü istisna yakalama veya bildirme

P2 metodunun aşağıdaki gibi tanımlandığını varsayalım:

```
void p2() throws IOException {
  if (a file does not exist) {
     throw new IOException("File does not exist");
  }

  ...
}
```



Kontrollü istisna yakalama veya bildirme

Java, kontrollü istisnalarla başa çıkmanızı sağlar. Bir yöntem kontrollü istisna (yani, Hata veya RuntimeException dışında bir istisna) bildirirse, bunu bir try-catch bloğunda çağırmanız veya istisnayı çağrı yöntemine atmak için bildirmeniz gerekir. Örneğin, p1 metodunun p2 metodunu çağırdığını varsayalım ve p2 kontrollü istisnayı atabilsin (e.g., IOException), Bu durumda kodu (a) veya (b) de gösterildiği gibi yazmanız gerekmektedir.

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

Örnek:
İstisna bildirimi, atma ve yakalama

Bu örnek, daha önce tanımlanan Circle sınıfında *setRadius* yöntemini değiştirerek özel durum bildirme, atma ve yakalama özelliklerini gösterir. Yeni *setRadius* yöntemi, yarıçap negatifse bir özel durum atar.

CircleWithException

TestCircleWithException Run



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

İstisnaları yeniden atma (Rethrowing Exceptions)

```
try {
  statements;
}
catch(TheException ex) {
  perform operations before exits;
  throw ex;
}
```



`finally` Cümleciği

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}
```



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Programın Yürütülmesi

İfadelerde hiç istisna olmadığını varsayalım

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```



Programın Yürütülmesi

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

final bloğu her zaman çalıştırılır



Programın Yürütülmesi

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Metodun sıradaki ifadesi çalıştırılır



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Exception1 türündeki bir istisananın satatement2 ifadesinde atıldığını varsayalım.



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

İstisna yönetildi



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

final bloğu her zaman çalıştırılır



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Metodun sıradaki ifadesi çalıştırıldı.



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Statement2, Exception2 türündeki bir istisnayı atsın



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

İstisna yönetimi



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Final bloğu çalıştırıldı



Programın Yürütülmesi

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

İstisna yeniden atılır ve kontrol ağrı yapana aktarılır.



İstisnaları Kullanırken Dikkat Edilecek Noktalar

- İstisna yönetimi, hata yönetimi (error-handling) kodunu normal programlama görevlerinden ayırır, böylece programların okunmasını ve değiştirilmesini kolaylaştırır. Bununla birlikte, istisna yönetiminin genellikle daha fazla zaman ve kaynak gerektirdiğinden, yeni bir istisna nesnesinin başlatılmasını, çağrı yığınının geri alınmasını ve hataların çağrı yöntemlerine yayılmasını gerektirdiğini unutmayın.



İstisnalar ne zaman atılmalı

- Bir yöntemde bir istisna oluşur. İstisnanın arayan tarafından işlenmesini istiyorsanız, bir istisna nesnesi oluşturmalı ve atmalısınız. İstisnayı yöntemde yönetebiliyorsanız, atmaya gerek yoktur.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

İstisnalar ne zaman kullanılmalı

Kodda try-catch bloğunu ne zaman kullanmalısınız? Beklenmedik hata durumlarıyla başa çıkmak için kullanmalısınız. Basit, beklenen durumlarla başa çıkmak için kullanmayın. Örneğin, aşağıdaki kod

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```



İstisnalar ne zaman kullanılmalı

Yerine aşağıdaki ifade kullanılabilir:

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```



Özel İstisna Sınıflarını Tanımlama

- Mümkün olduğunca API'daki istisna sınıflarını kullanın.
- Önceden tanımlanmış sınıflar yeterli değilse özel durum sınıflarını tanımlayın.
- İstisna veya İstisna alt sınıfını genişleterek özel istisna sınıflarını tanımlayın



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Özel İstisna Sınıfı Örneği

Liste 13.8'de, yarıçap negatifse setRadius yöntemi bir istisna atar. Yarıçapı handler'a geçirmek istediğinizi varsayalım, özel bir istisna sınıfı oluşturmanız gerekir.

InvalidRadiusException

CircleWithRadiusException

TestCircleWithRadiusException

Run



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

TEXT I/O



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

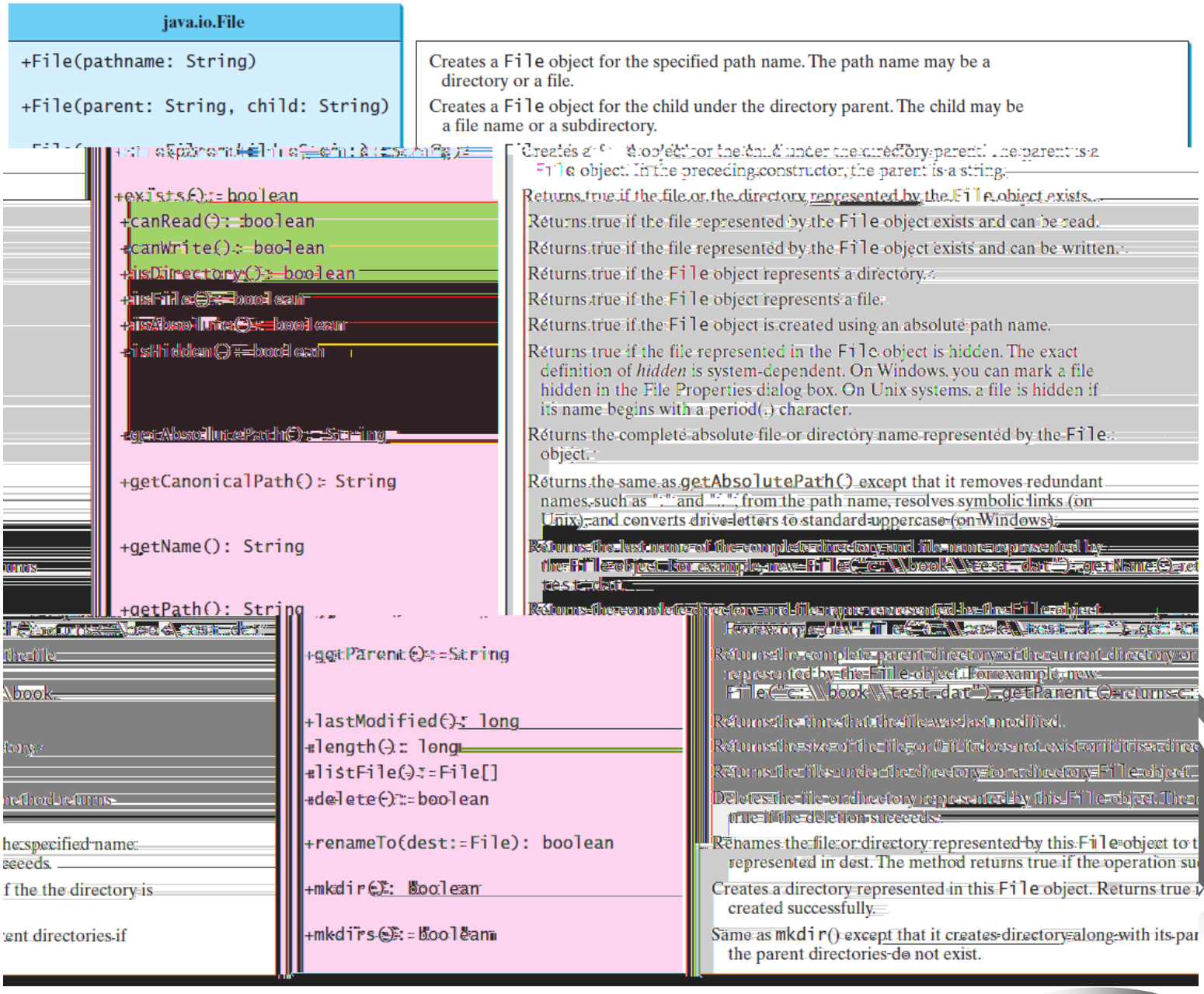
Dosya Sınıfı (The File Class)

File sınıfının, makineye bağımlı dosya ve yol adlarının makineye bağlı karmaşıklıklarının çoğunu, makineden bağımsız bir şekilde ele alan bir soyutlama sağlaması amaçlanmıştır. Dosya adı bir dizedir. File sınıfı, dosya adı ve dizin yolu için bir sarmalayıcı sınıfıdır.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Dosya özelliklerini alma ve dosyayı değiştirme



Dosya Sınıfı (The File Class)

```
import java.io.File;

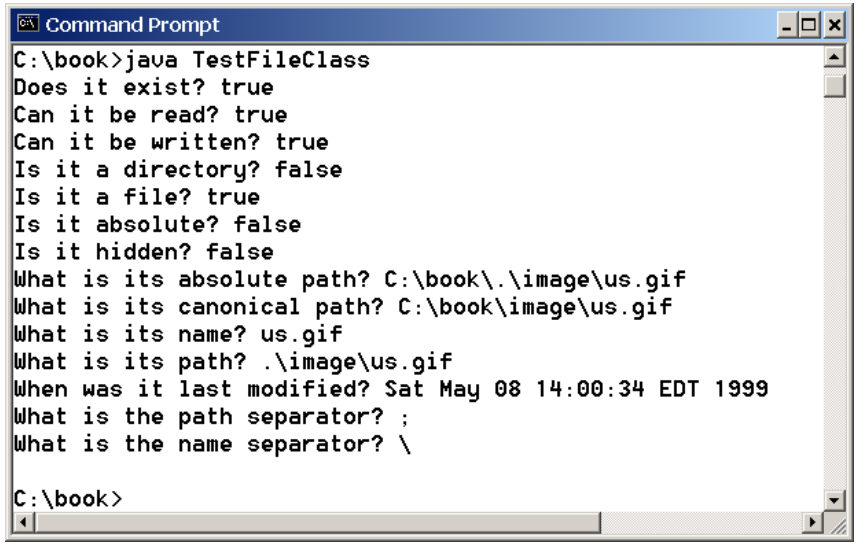
File dosya = new File(dAdi);
dosya
dosya.getAbsolutePath()
dosya.getPath()
dosya.getName()
dosya.getParent()
dosya.exists()
dosya.canRead()
dosya.canWrite()
dosya.isDirectory()
dosya.isFile()
dosya.lastModified()
dosya.length()
```

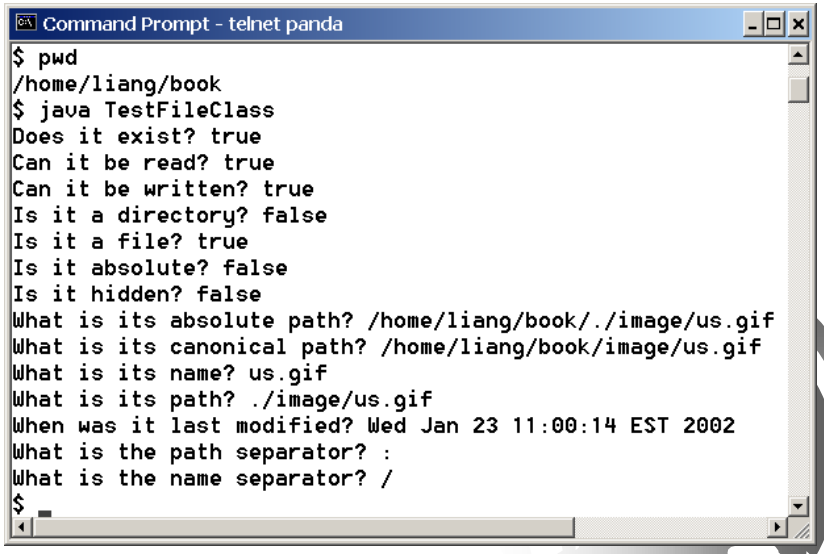


Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Problem: Dosya Özelliklerini Keşfedin

Amaç: Dosyaların platformdan bağımsız bir şekilde nasıl oluşturulacağını gösteren bir program yazın ve özelliklerini elde etmek için File sınıfındaki yöntemleri kullanın. Aşağıdaki şekiller, Windows ve Unix'te programın örnek bir çalışmasını gösterir.





TestFileClass Run

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Metin I/O (Text I/O)

Bir File nesnesi, bir dosyanın veya yolun özelliklerini içerir, ancak bir dosyadan/dosyaya veri okuma/yazma yöntemlerini içermez. I/O gerçekleştirmek için uygun Java I/O sınıflarını kullanarak nesne oluşturmanız gerekir. Nesneler, bir dosyadan/dosyaya veri okuma/yazma yöntemlerini içerir. Bu bölümde, Scanner ve PrintWriter sınıflarını kullanarak bir metin dosyasından dizelerin ve sayısal değerlerin nasıl okunacağı/yazılacağı anlatılmaktadır.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

PrintWriter kullanarak veri yazma



WriteData

Run

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Try-with-resources

Programcılar genellikle dosyayı kapatmayı unutur. JDK 7, dosyaları otomatik olarak kapatan try-with-resources (yeni kaynaklar denemesi) sözdizimi sağlar.

```
try (declare and create resources) {
  Use the resource to process the file;
}
```

WriteDataWithAutoClose

Run



Scanner kullanarak veri okuma

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner object to read data from the specified file.
+Scanner(source: String)	Creates a Scanner object to read data from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

ReadData Run

Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Problem: Metin değiştirme

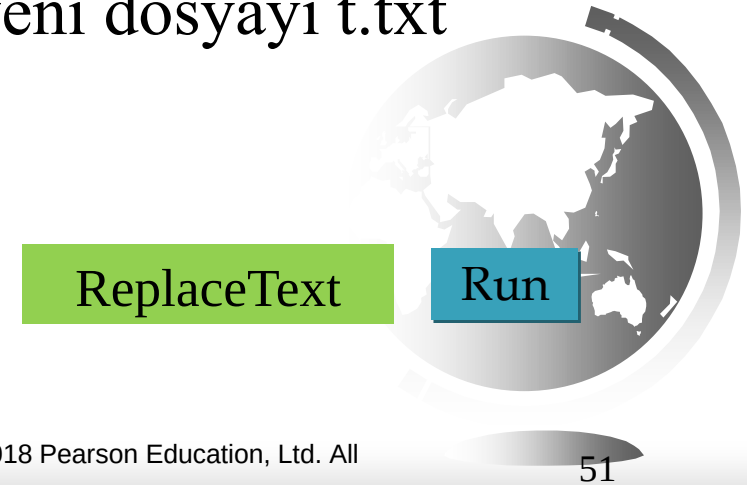
Metin dosyasındaki bir dizeyi yeni bir dizeyle değiştiren ReplaceText adlı bir sınıf yazın. Dosya adı ve dizeler aşağıdaki gibi komut satırı bağımsız değişkenleri olarak iletilir:

```
java ReplaceText sourceFile targetFile oldString newString
```

Örneğin aşağıdaki ifade,

```
java ReplaceText FormatString.java t.txt StringBuilder StringBuffer
```

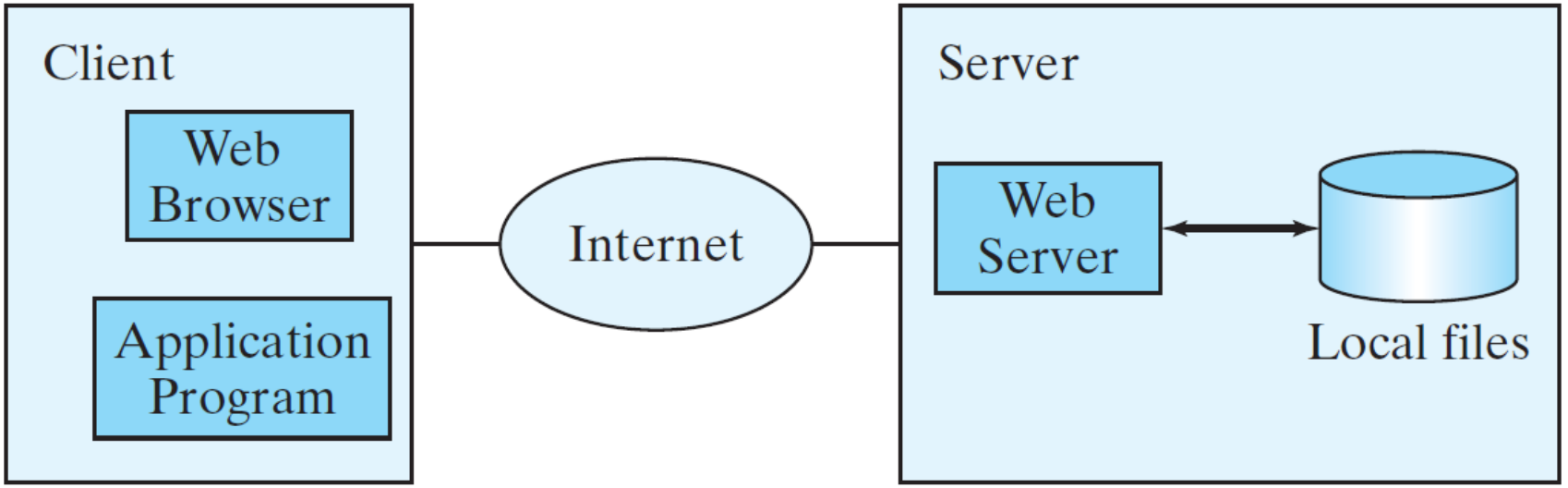
FormatString.java içinde StringBuilder tarafından tüm StringBuffer örneklerini değiştirir ve yeni dosyayı t.txt dosyasına kaydeder.



Liang, Introduction to Java Programming, Eleventh Edition, (c) 2018 Pearson Education, Ltd. All rights reserved.

Web’den veri okuma

Bilgisayarınızdaki bir dosyadan veri okuyabileceğiniz gibi, Web'deki bir dosyadan da veri okuyabilirsiniz.



Web'den veri okuma

URL url = **new** URL(**"www.google.com/index.html"**);

Bir URL nesnesi oluşturulduktan sonra, bir giriş akışını açmak için URL sınıfında tanımlanan openStream () yöntemini kullanabilir ve aşağıdaki gibi bir Tarayıcı nesnesi oluşturmak için bu akışı kullanabilirsiniz:

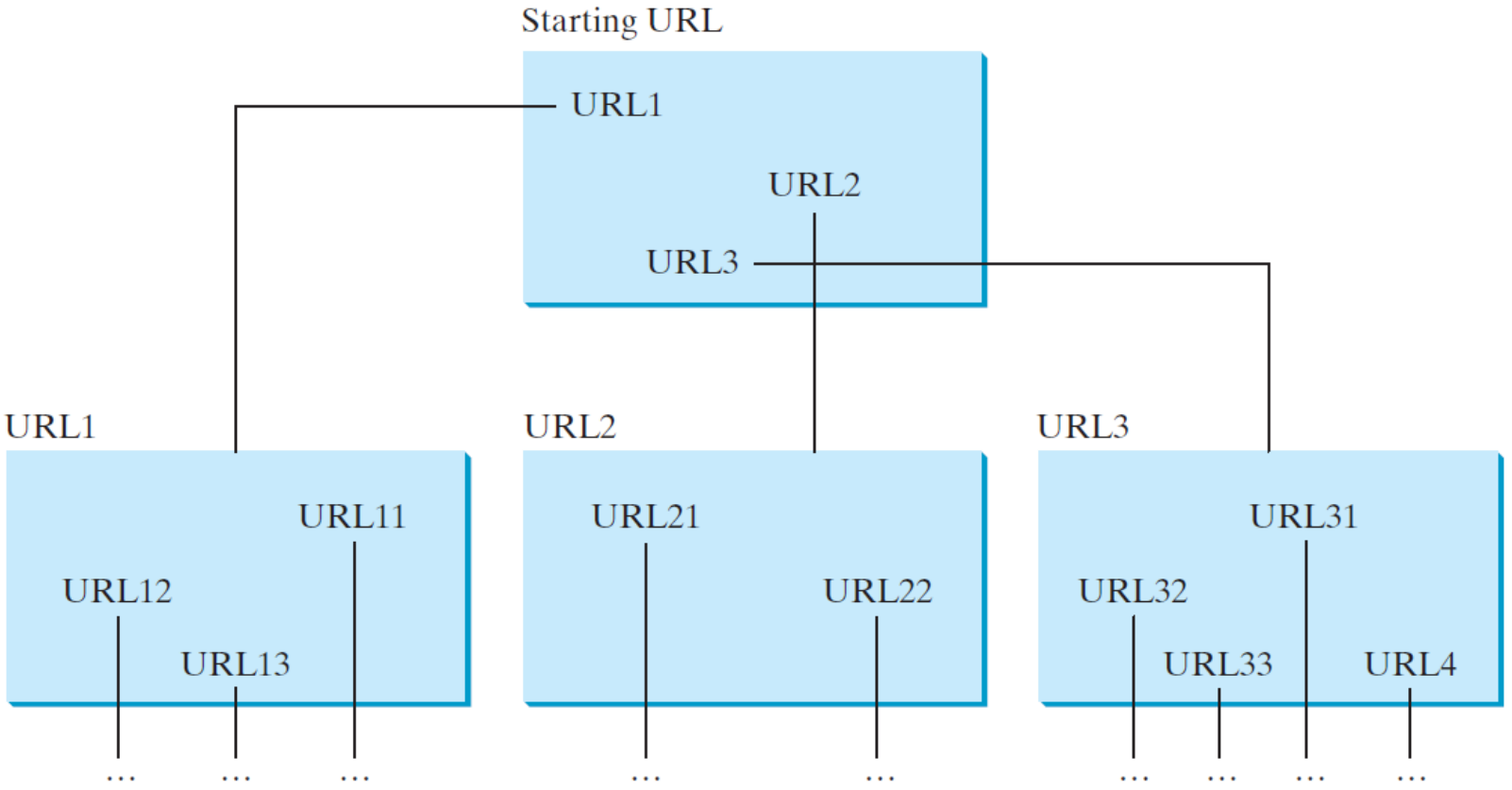
Scanner input = **new** Scanner(url.openStream());

ReadFileFromURL Run



Case Study: Web Crawler

Bu çalışma, hyperlink'leri takip ederek Web'de dolaşan bir program geliştirir.



Case Study: Web Crawler



Case Study: Web Crawler

```
Add the starting URL to a list named listOfPendingURLs;
while listOfPendingURLs is not empty {
      Remove a URL from listOfPendingURLs;
      if this URL is not in listOfTraversedURLs {
        Add it to listOfTraversedURLs;
        Display this URL;
        Exit the while loop when the size of S is equal to 100.
        Read the page from this URL and for each URL contained in the page {
          Add it to listOfPendingURLs if it is not is listOfTraversedURLs;
        }
    }
}
```

WebCrawler Run

